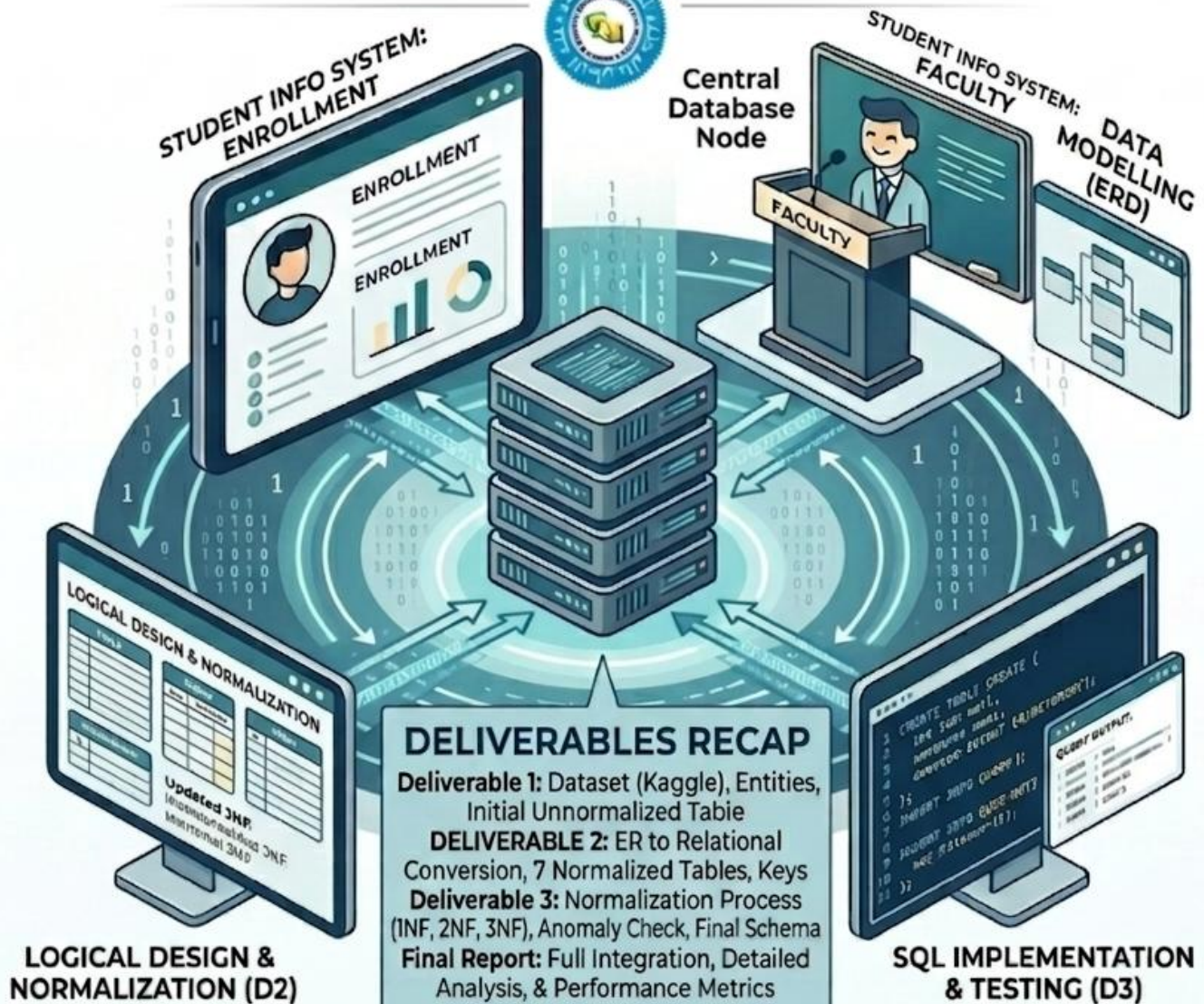


FAST UNIVERSITY



DATABASE PROJECT REPORT FOR: STUDENT INFORMATION SYSTEM Covering D2 Logical Design & D3 SQL Implementation

SUBMITTED BY (Group Members):

Syed Munsab (23L-5102) Ali Naveed (23L-5094)
Hareem Jawad (23L-5051)

SUBMITTED TO (Instructor):

Sir Aaib Zeeshan

Contents

Deliverable 1	1
Problem Statement	1
Objectives	1
Scope.....	1
Student Records	2
Department Management	2
Course Management	2
Enrollment Tracking	2
Grade Management	2
Attendance Monitoring	2
Requirements	3
Data Operations (Functional Requirements)	3
Business Rules	3
Student Information System (Core Entities)	3
Student	3
Department	4
Course	4
Instructor	5
Enrollment	5
Grade	5
Attendance	6
Define Relationships	6
Department & Student	6
Department & Course	6
Student & Enrollment	6
Course & Enrollment	7
Instructor & Department	7
Enrollment & Grade.....	7
Student & Attendance	7
Course & Attendance	7
Entity Relationship Diagram	8
Deliverable 2	9

Dataset Selection & Loading	9
Starting Point: The Flat Unnormalized Table	10
Transformation Rules Applied	12
Rule 1	12
Rule 2.....	12
Rule 3	13
Rule 4.....	13
Result From 1 Flat Table to 7 Normalized Tables	13
Normalization (1NF → 2NF → 3NF)	15
First Normal Form (1NF)	15
Second Normal Form (2NF)	16
Third Normal Form (3NF)	16
Screenshot 1: Dataset Source Page	18
Screenshot 2: Data Loaded in SSMS	19
Screenshot 3: SELECT Query Running	23
Deliverable 3	24
SQL Implementation & Testing Database Implementation Script	24
SQL Queries	25
Advanced Feature	31
Testing Results	31

Acknowledgement

*We are grateful to Allah Almighty for providing us with health to complete this project.
We express our gratitude to Sir Aqib Zehehsan and TA Zainab Fatima Aroosa. Our course
instructor for his guidance, wisdom and inspiration. We appreciate all his assistance. We also
thank everyone who contributed to our work.
We are grateful to everyone!*

Deliverable 1

Problem Statement

Numerous universities and education institutions around the world have been using old fashioned and manual means of handling very important academic information. The student records are kept in disjointed spread sheets or hardcopy files, course enrollment is done using paper documents, grades are computed manually and stored in spread sheets, attendance in registers and so on. This is not only time consuming, but also very prone to errors. Information is copied, misplaced, and is not consistent, interdepartmentally. Executive personnel waste colossal sums of time in activities that can be automated. Faculty members find it difficult to get real and current information about students. Most importantly, the decision-makers like the deans and the department heads are unable to obtain trustworthy academic performance reports whenever they require it. The essence of the matter is that there is no centralized, structured, and queryable system that would incorporate all the academic data on students under the same roof and make them accessible, consistent, and easily manageable.

Objectives

The main aim of the project is to design and deploy a complete working relational database system, which will computerize and centralize academic activities of a university. In particular, the system will seek to:

- Replace traditional manual record keeping by putting all student related data, course related data and instructor related data as well as departmental specific data in structured relational tables.
- Guarantee data integrity by provision of primary keys, foreign keys, and constraints to ensure that, there are no invalid records or orphaned records in the system.
- Facilitate rapid and precise search of scholarly data using SQL query with complex multi-table joins and aggregate functions.
- Streamline the monitoring of student enrollments over a number of different semesters and academic years.
- Automate the grading system based on the marks calculated due to enrollment in the course.
- Track attendance of students at the course level and indicate students with low attendance.
- Lighten administrative burden through the substitution of manual operations with database-based operations.
- Develop a database platform that can be later expanded to accommodate applications such as GPA computation, fee processing and online portals.

Scope

The area of my project encompasses the fundamental university business of a medium-sized university. The system will have the capacity to deal with the following six functional areas:

Student Records

Personal and academic profiles of individual students are fully stored within the system. This contains their complete name, date of birth, gender, contact information, address in the place of residence, department in which they are, and admission year. All the students are given an unique individual Student ID which is used to identify the student in all the other sections of the system.

Department Management

All the academic departments of the university are registered in the system as well as their name and the name of the Head of Department. The main unit of organization is the department that includes students, instructors, and courses. This will enable the system to produce departmental reports and statistics.

Course Management

The system is used to maintain a record of the courses available in the university. Every course bears a course code, a descriptive course name, the number of credit hours associated with it, the department within which it falls and the instructor that will teach the course. This simplifies the process of getting to know what is being offered every semester and whoever is in charge of any course.

Enrollment Tracking

Student enrolment is managed by a special enrolment table that contains a list of student-courses enrolments per semester and academic year telling us about the student who took what course during which semester and academic year. This many to many relationship is addressed well by the enrollment entity since a student can take a large number of courses and a course can take a large number of students. This table is the main point of connection between students and their academic activity.

Grade Management

The grades are kept in the level of enrollment where a record of the grades can be generated every time a student enrolls to a course. The system will have both the numerical marks scored by the student and the letter grade (A, B, C, D, or F). It may be provided as a database trigger which may automatically place the letter grade based on the marks provided in it eliminating the chances of any human error in the classification of the grade.

Attendance Monitoring

The system captures attendance of students on a daily basis on a course to course basis. The attendance record records the student, the course, the date and the student was Present, Absent or Late. This enables the faculty to track attendance and the administration to know the at-risk students because of low attendance before it can impact their academic status. Furthermore It is important that the system aligns with university criteria of attendance percentage and automatically flag students below the minimum requirement.

Requirements

Data Operations (Functional Requirements)

These are a description of the basic operations your system needs to provide:

- The system should have the capability of adding, updating and deleting student records.
- The system should have the ability to store and control the department details.
- The system should enable the instructors to be connected to the departments and courses.
- The system should record the enrolments of the students in courses per semester/year.
- The system should store grades by each enrollment and it should be able to compute letter grades.
- The system should monitor the attendance of each student on a course session basis.
- The system should produce reports (e.g. student transcripts, attendance summaries).

Business Rules

These are data integrity constraints or data integrity rules:

- Every student is expected to be a member of a single department.
- All courses should be provided by one department.
- A teacher is allowed to teach more than one course though he/she has to be a member of a single department.
- A student is allowed to take several courses but no two records of enrollment should have the identical studentID + CourseID + Semester + Year.
- There should be a maximum of one grade record to each enrollment.
- There has to be a connection of attendance records with valid students and courses.
- Primary keys should be a unique identifier of each entity and foreign keys should establish a relationship.

Student Information System (Core Entities)

Student

Attribute	Description
StudentID (PK)	Unique identifier for each student
Name	Full name of the student

DOB	Date of birth
Email	Contact email address
Phone	Contact phone number
DepartmentID (FK)	Links student to their department

Department

Attribute	Description
DeptID (PK)	Unique identifier for each department
DeptName	Name of the department
HOD	Head of Department

Course

Attribute	Description
CourseID (PK)	Unique identifier for each course
CourseName	Name of the course
Credits	Credit hours assigned to the course
DeptID (FK)	Links course to its department

Instructor

Attribute	Description
InstructorID (PK)	Unique identifier for each instructor
Name	Full name of the instructor
Email	Contact email address
DeptID (FK)	Links instructor to their department

Enrollment

Attribute	Description
EnrollmentID (PK)	Unique identifier for each enrollment record
StudentID (FK)	Links enrollment to a student
CourseID (FK)	Links enrollment to a course
Semester	Semester in which the course is taken
Year	Academic year

Grade

Attribute	Description
GradeID (PK)	Unique identifier for each grade record
EnrollmentID (FK)	Links grade to enrollment record

Marks	Numerical marks obtained
LetterGrade	Letter grade (A, B, C, etc.)

Attendance

Attribute	Description
AttendanceID (PK)	Unique identifier for each attendance record
StudentID (FK)	Links attendance to a student
CourseID (FK)	Links attendance to a course
Date	Date of the class session
Status	Attendance status (Present/Absent)

Define Relationships

Department & Student

Department (1) ——— (Many) Student

Relationship: One-to-Many.

One department can have many students, but a student belongs to at most one department.

Department & Course

Department (1) ——— (Many) Course

Relationship: One-to-Many.

Each department offers multiple courses, while each course is tied to one specific department.

Student & Enrollment

Student (1) ——— (Many) Enrollment

Relationship: One-to-Many.

A single student can have multiple enrollment records (for different courses/semesters).

Course & Enrollment

Course (1) — (Many) Enrollment

Relationship: One-to-Many.

A course can have many students enrolled in it.

Instructor & Department

Department (1) — (Many) Instructor

Relationship: One Department can have many Instructors, Each Instructor belongs to one Department.

Enrollment & Grade

Enrollment (1) — (1) Grade

Relationship: One-to-One.

Each specific enrollment record typically results in one final grade.

Student & Attendance

Student (1) — (Many) Attendance

Relationship: One-to-Many.

A student has multiple attendance records over time.

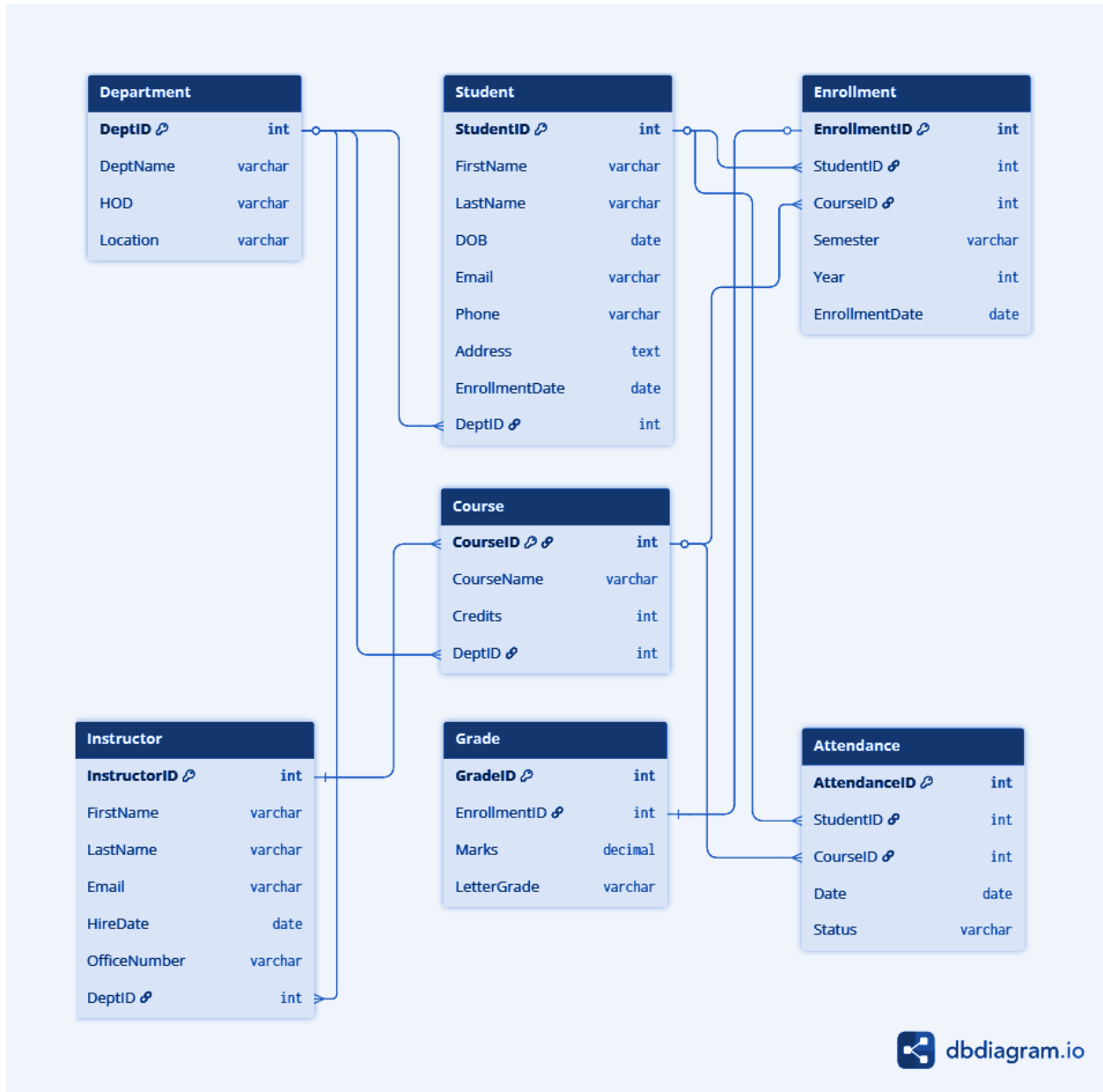
Course & Attendance

Course (1) — (Many) Attendance

Relationship: One-to-Many.

A course generates multiple attendance entries for its various sessions.

Entity Relationship Diagram



Link to the diagram

<https://dbdiagram.io/d/69b10a7477d079431b565057>

Deliverable 2

Dataset Selection & Loading

Continuing from our last deliverable, where we selected the Student Information System and laid the foundation for database work, we now move on to Deliverable 2. For this part, we identified and selected an appropriate dataset from Kaggle **Student Performance & Behavior Dataset**. The dataset

which matched almost all the entities defined in our ER diagram. This dataset proved to be a perfect fit for our Student Information System, as it contained grades, attendance, and students data across multiple departments. However, this dataset was also focusing on students behaviour we had to drop those columns as that was not the aim of our student information system.

Some entries that were missing in the original dataset were generated artificially, ensuring a balanced approach that combined real-world data with controlled generation. This allowed us to maintain consistency across all entities while preserving the integrity of the dataset. Importantly, the dataset was clean, easy to work with, and contained no errors, making it highly suitable for our project.

The dataset was downloaded in .csv format and imported into SQL Server Management Studio (SSMS) as a flat table. It was then distributed into 7 normalized tables aligned with the ER diagram from Deliverable 1, covering the following entities:

- Student
- Department
- Instructor
- Course
- Enrollment
- Grade
- Attendance

The flat imported file contained 500 rows and 30 columns, comprehensively covering all required entities and relationships.

Relational Schema

Every entity and relationship from the ER diagram has been converted into the following relational tables. Primary Keys (PK) and Foreign Keys (FK) are marked for each table.

Table 1

department (deptid INT PK, deptname VARCHAR(100), hod VARCHAR(100), location VARCHAR(50))

student (studentid INT PK, firstname VARCHAR(50), lastname VARCHAR(50), dob DATE,

email VARCHAR(100), phone VARCHAR(25), address VARCHAR(255),
enrollmentdate DATE, deptid INT FK → department)

instructor (instructorid INT PK, firstname VARCHAR(50), lastname VARCHAR(50),

email VARCHAR(100), hiredate DATE, officenumber VARCHAR(20),
deptid INT FK → department)

course (courseid INT PK, coursename VARCHAR(100), credits INT,

deptid INT FK → department)

enrollment (enrollmentid INT PK, studentid INT FK → student,

courseid INT FK → course, semester VARCHAR(20), year INT,
enrollmentdate DATE)

grade (gradeid INT PK, enrollmentid INT FK → enrollment,

marks DECIMAL(5,2), lettergrade VARCHAR(2))

attendance (attendanceid INT PK, studentid INT FK → student,

courseid INT FK → course, attendancedate DATE, status VARCHAR(10))

Transformation Steps

ER Diagram → Relational Tables

Starting Point: The Flat Unnormalized Table

Before applying any transformation rules, all the data existed as one single flat table. This flat table was imported from the Kaggle dataset and contained 500 rows and 30 columns. Every piece of

information about a student, their department, their instructor, their course, their grades, and their attendance was all stored in one single row. This is called an unnormalized table.

The flat table contained these columns all in one place:

Table 2

Category	Columns in the Flat Table
Student Info	StudentID, FirstName, LastName, DOB, Email, Phone, Address, EnrollmentDate
Department Info	DeptID, DeptName, HOD, Location
Instructor Info	InstructorID, Instructor_FirstName, Instructor_LastName, Instructor_Email, HireDate, OfficeNumber
Course Info	CourseID, CourseName, Credits
Enrollment Info	EnrollmentID, Semester, Year
Grade Info	GradeID, Marks, LetterGrade
Attendance Info	AttendanceID, AttendanceDate, AttendanceStatus

Problems identified in this flat table:

- DeptName, HOD, and Location repeated for every single student even though department data never changes.
- Instructor data repeated hundreds of times across all students belonging to the same department.
- No clear separation between data belonging to different real world entities.
- Updating one department name would require updating hundreds of rows causing data inconsistency.
- The table had no structure as student data, course data, and grade data were all mixed together.

Transformation Rules Applied

To fix the problems in the flat table, the following ER to relational transformation rules were applied. Each rule is explained below along with which entity or relationship it was applied to and what the result was.

Rule 1

Strong Entity becomes its own table

A strong entity is one that exists independently and has its own primary key. It does not depend on any other entity to identify itself. Each strong entity from the ER diagram was converted into its own separate table.

- **Department** existed independently in the ER diagram. It became its own table with DeptID as the primary key. The columns DeptName, HOD, and Location were moved here. This eliminated the repetition of department data that appeared in every single student row in the flat table.
- **Student** existed independently. It became its own table with StudentID as the primary key. The columns FirstName, LastName, DOB, Email, Phone, Address, and EnrollmentDate belong to this table.
- **Instructor** existed independently. It became its own table with InstructorID as the primary key. The columns FirstName, LastName, Email, HireDate, and OfficeNumber belong here.
- **Course** existed independently. It became its own table with CourseID as the primary key. The columns CourseName and Credits belong here.

Rule 2

Weak Entity becomes its own table with a Foreign Key to its owner

A weak entity cannot exist on its own. It depends on another entity to give it meaning. Without its owner entity, a weak entity record has no purpose. The following were identified as weak entities in the ER diagram:

- **Grade** is a weak entity because a grade has no meaning unless we know which enrollment it belongs to. You cannot have a grade without an enrollment. It became its own table with GradeID as the primary key and EnrollmentID as a foreign key referencing the Enrollment table.
- **Attendance** is a weak entity because an attendance record only makes sense when we know which student attended which course. It became its own table with AttendanceID as the primary key and StudentID and CourseID as foreign keys.

Rule 3

M:N Relationship becomes a Junction Table

A many to many relationship cannot be stored with just a foreign key on one side. In the ER diagram, Student and Course had a many to many relationship, one student can enroll in many courses and one course can have many students. This type of relationship must be broken down into its own separate table called a junction table or bridge table.

- **Enrollment** was not originally an entity in the ER diagram as it was the M:N relationship between Student and Course. It was converted into its own junction table with EnrollmentID as the primary key and StudentID and CourseID as foreign keys pointing to both sides of the relationship. Semester, Year, and EnrollmentDate were added as descriptive attributes of this relationship.

Rule 4

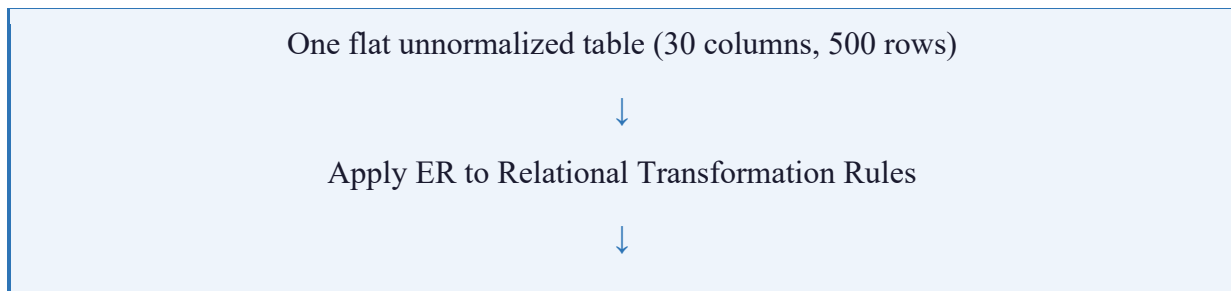
1:N Relationship places Foreign Key on the Many side

When one entity is related to many instances of another entity, the foreign key is placed on the many side of the relationship. This is the most common rule in relational design.

- One Department has many Students → DeptID placed as foreign key in Student table
- One Department has many Instructors → DeptID placed as foreign key in Instructor table
- One Department has many Courses → DeptID placed as foreign key in Course table
- One Enrollment has one Grade → EnrollmentID placed as foreign key in Grade table

Result From 1 Flat Table to 7 Normalized Tables

After applying all transformation rules, the single flat table was broken down into 7 clean separate tables. Each table now stores only the data that belongs to it and connects to other tables through primary keys and foreign keys.



7 clean normalized tables linked by PKs and FKs

Department → Student, Instructor, Course

Student + Course → Enrollment

Enrollment → Grade

Student + Course → Attendance

Summary of all 7 tables after transformation:

Table 3

Table	Source in Flat File	Type	Primary Key	Foreign Keys
Department	DeptID, DeptName, HOD, Location	Strong Entity	DeptID	None
Student	StudentID, FirstName, LastName, DOB, Email, Phone, Address	Strong Entity	StudentID	DeptID → Department
Instructor	InstructorID, FirstName, LastName, Email, HireDate, OfficeNumber	Strong Entity	InstructorID	DeptID → Department
Course	CourseID, CourseName, Credits	Strong Entity	CourseID	DeptID → Department
Enrollment	EnrollmentID, Semester, Year, EnrollmentDate	M:N Relationship	EnrollmentID	StudentID → Student, CourseID → Course
Grade	GradeID, Marks, LetterGrade	Weak Entity	GradeID	EnrollmentID → Enrollment

Table	Source in Flat File	Type	Primary Key	Foreign Keys
Attendance	AttendanceID, AttendanceDate, Status	Weak Entity	AttendanceID	StudentID → Student, CourseID → Course

Normalization (1NF → 2NF → 3NF)

First Normal Form (1NF)

A table is in 1NF if every column holds a single atomic value, there are no repeating groups, and every row is uniquely identifiable.

Functional Dependencies:

```

student:  studentid → firstname, lastname, dob, email, phone, address, enrollmentdate,
deptid
department: deptid → deptname, hod, location
instructor: instructorid → firstname, lastname, email, hiredate, officenumber, deptid
course:  courseid → coursename, credits, deptid
enrollment: enrollmentid → studentid, courseid, semester, year, enrollmentdate
grade:  gradeid → enrollmentid, marks, lettergrade
attendance: attendanceid → studentid, courseid, attendancedate, status

```

Violations Found:

The original flat dataset had all 30 columns in one row. Columns like DeptName, HOD, and Location were repeating the same department data for every student meaning this is a repeating group violation.

Fix Applied:

Department data was separated into its own table. Each attribute now holds a single atomic value. All tables have a single-column primary key making every row uniquely identifiable. No column holds multiple values.

Result: All tables satisfy 1NF.

Second Normal Form (2NF)

A table is in 2NF if it satisfies 1NF and has no partial dependencies. A partial dependency means a non-key attribute depends on only part of a composite primary key.

Check Applied:

All seven tables in this schema use a single-column primary key (StudentID, DeptID, InstructorID, CourseID, EnrollmentID, GradeID, AttendanceID). Partial dependencies can only exist when there is a composite primary key. Since no table here has a composite key, partial dependencies cannot exist.

Result: All tables automatically satisfy 2NF.

Third Normal Form (3NF)

A table is in 3NF if it satisfies 2NF and has no transitive dependencies. A transitive dependency means a non-key attribute depends on another non-key attribute instead of directly on the primary key.

Transitive Dependency 1:

Before (violation):

student table contained: studentid, firstname, ..., deptid, deptname, hod, location

$\text{studentid} \rightarrow \text{deptid} \rightarrow \text{deptname, hod, location}$

deptname, hod, location depend on deptid (non-key), not on studentid directly.

this is a transitive dependency.

After (fixed):

student (studentid PK, firstname, lastname, dob, email, phone, address,
enrollmentdate, deptid FK)

department (deptid PK, deptname, hod, location)

deptname, hod, location now depend directly on deptid which is the PK of the department table. transitive dependency removed.

Transitive Dependency 2:

Before (violation):

course table could have contained: courseid, coursename, credits, deptid, deptname

courseid $\rightarrow$ deptid $\rightarrow$ deptname

deptname depends on deptid (non-key), not on courseid directly.

After (fixed):

course (courseid PK, coursename, credits, deptid FK)

department (deptid PK, deptname, hod, location)

deptname now lives only in department table. removed from course table.

Remaining Tables No Transitive Dependencies:

Grade: marks and lettergrade both depend only on gradeid. Neither depends on the other.

Attendance: all attributes depend only on attendanceid directly.

Enrollment: semester, year, enrollmentdate all depend only on enrollmentid directly.

Instructor: all attributes depend directly on instructorid. DeptID is a FK not a transitive chain.

Result: All tables satisfy 3NF.

Final Schema Diagram

The final normalized schema diagram has been created using dbdiagram.io and is embedded below. The diagram shows all 7 tables with their attributes, primary keys, foreign keys, and connecting lines between related tables.

Screenshots

Screenshot 1: Dataset Source Page

The screenshot shows the Kaggle dataset page for 'Student Performance & Behavior Dataset' by MAHMOUD ELHEMALY. The page includes a sidebar with navigation links, a search bar, and a main content area with tabs for 'Data Card', 'Code (136)', 'Discussion (11)', and 'Suggestions (0)'. The 'About Dataset' section describes the dataset as real data of 5,000 records collected from a private learning provider. The 'Columns' section lists two columns: 'Student_ID' and 'First_Name'. The right sidebar shows 'Usability' (8.24), 'License' (Other (specified in...)), 'Expected update' (Never), and 'Tags' (Earth and Nature).

https://www.kaggle.com/datasets/mahmoudelhemy/students-grading-dataset

Install ☆

Search

MAHMOUD ELHEMALY · UPDATED A YEAR AGO

268 Code Download

Student Performance & Behavior Dataset

Students Grading Analysis

Data Card Code (136) Discussion (11) Suggestions (0)

About Dataset

Student Performance & Behavior Dataset

This dataset is real data of 5,000 records collected from a private learning provider. The dataset includes key attributes necessary for exploring patterns, correlations, and insights related to academic performance.

Columns:

1. Student_ID: Unique identifier for each student.
2. First_Name: Student's first name.

Usability ⓘ
8.24

License
Other (specified in...)

Expected update
Never

Tags
Earth and Nature

Figure 1

10 of 23 columns

 **Suggest Edits**

Version 5 (8.05 MB)

- ## Summary

-

- | | |
|----|--------|
| 10 | Summer |
| 11 | Fall |
| 12 | Winter |

- | | | |
|--|----|--------|
| | 23 | Fall |
| | 24 | Summer |
| | 25 | Summer |

- | | |
|----|----------|
| 37 | Full |
| 38 | Survivor |
| 39 | Full |

Screenshot 2: Data Loaded in SSMS

Figure 3

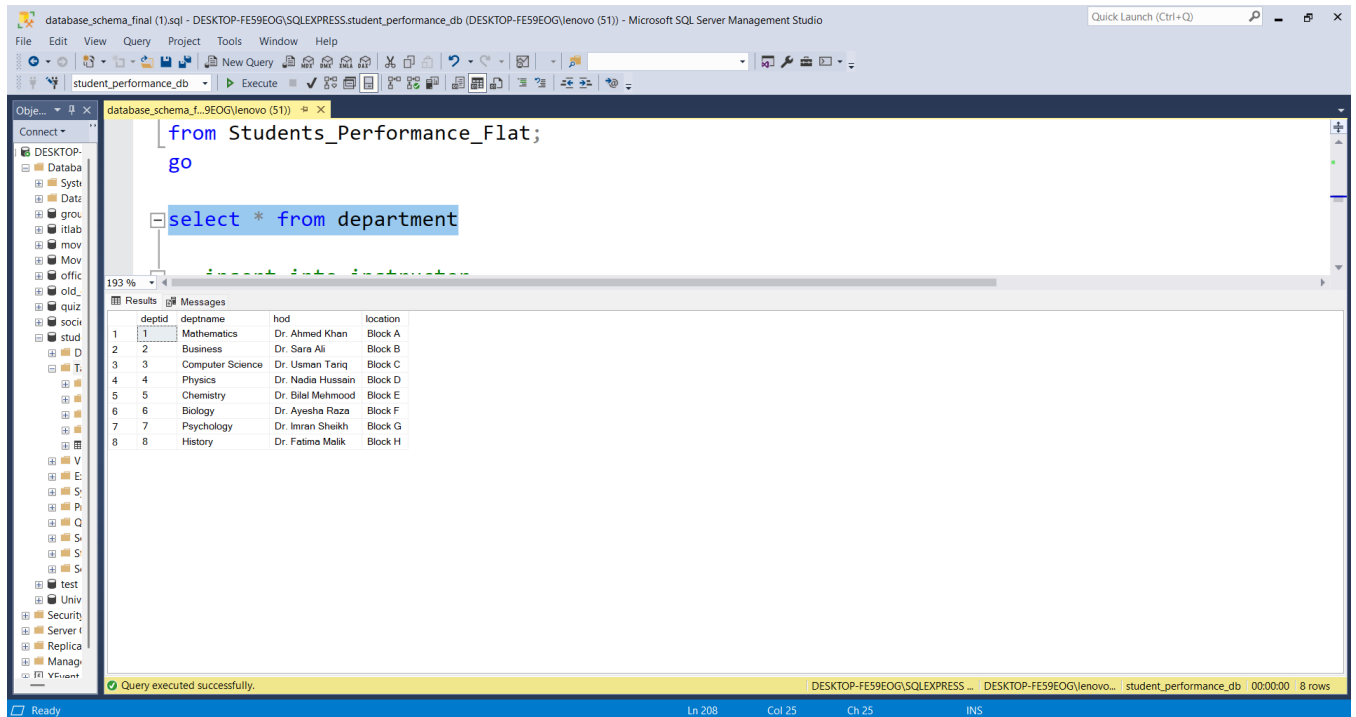


Figure 4

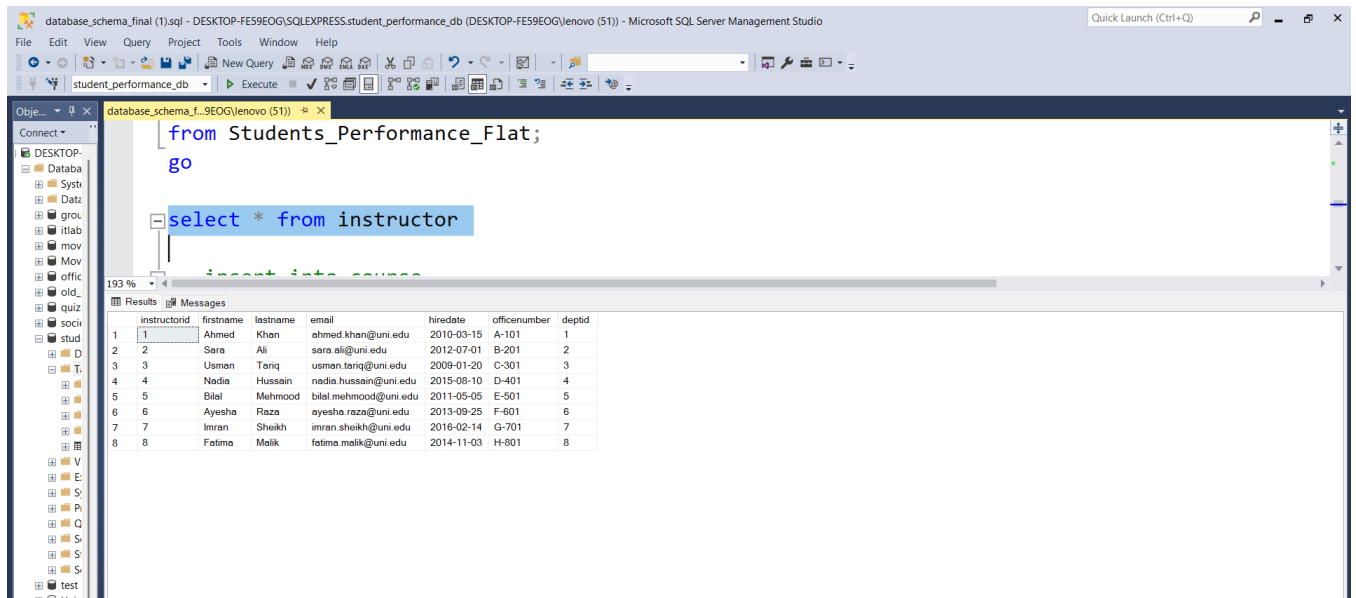


Figure 5

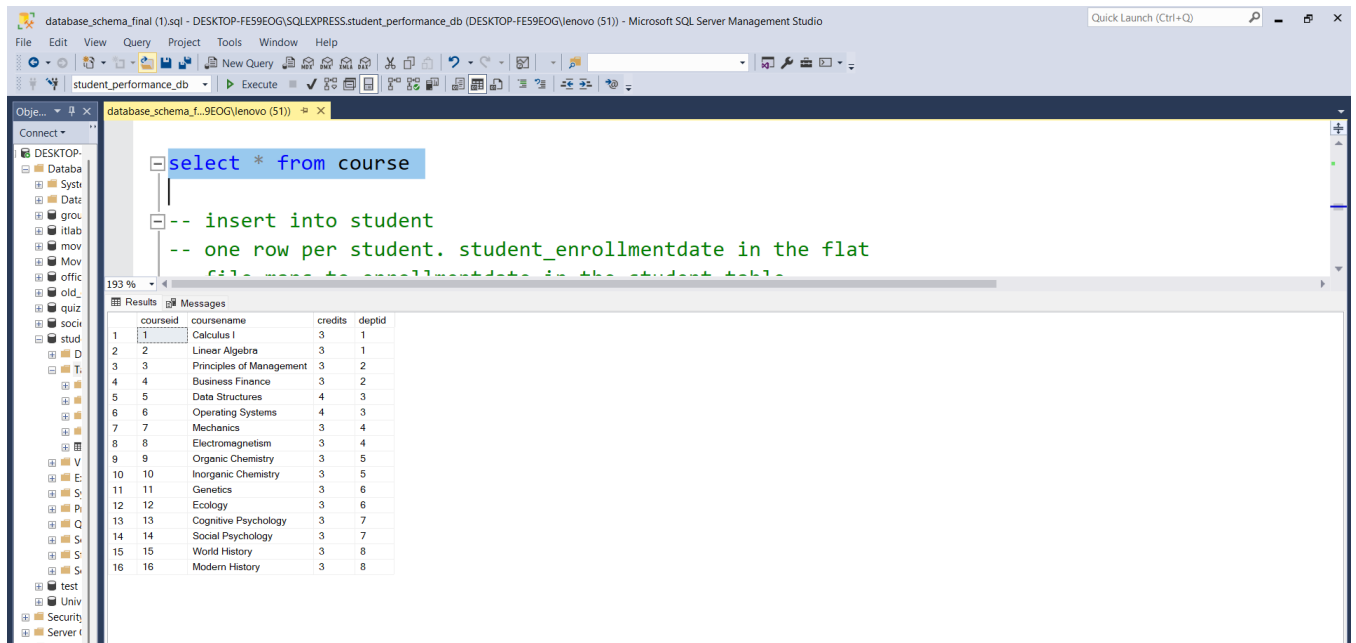


Figure 6

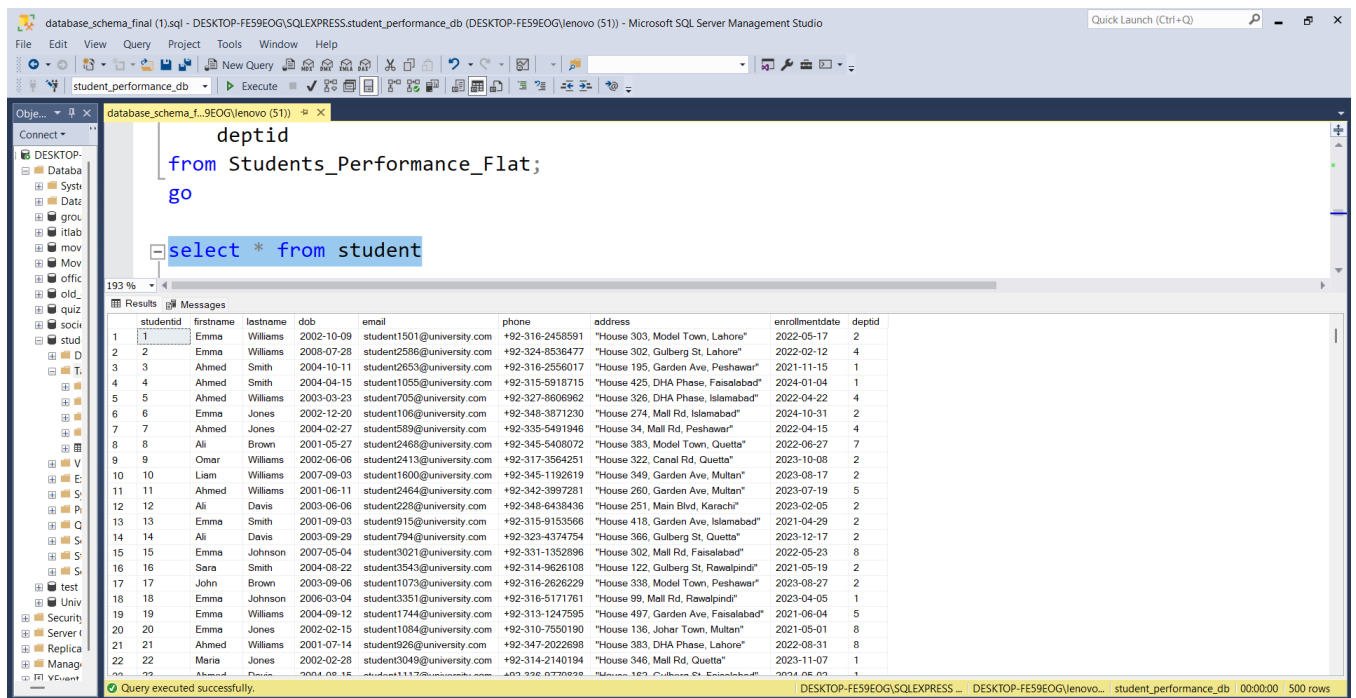


Figure 7

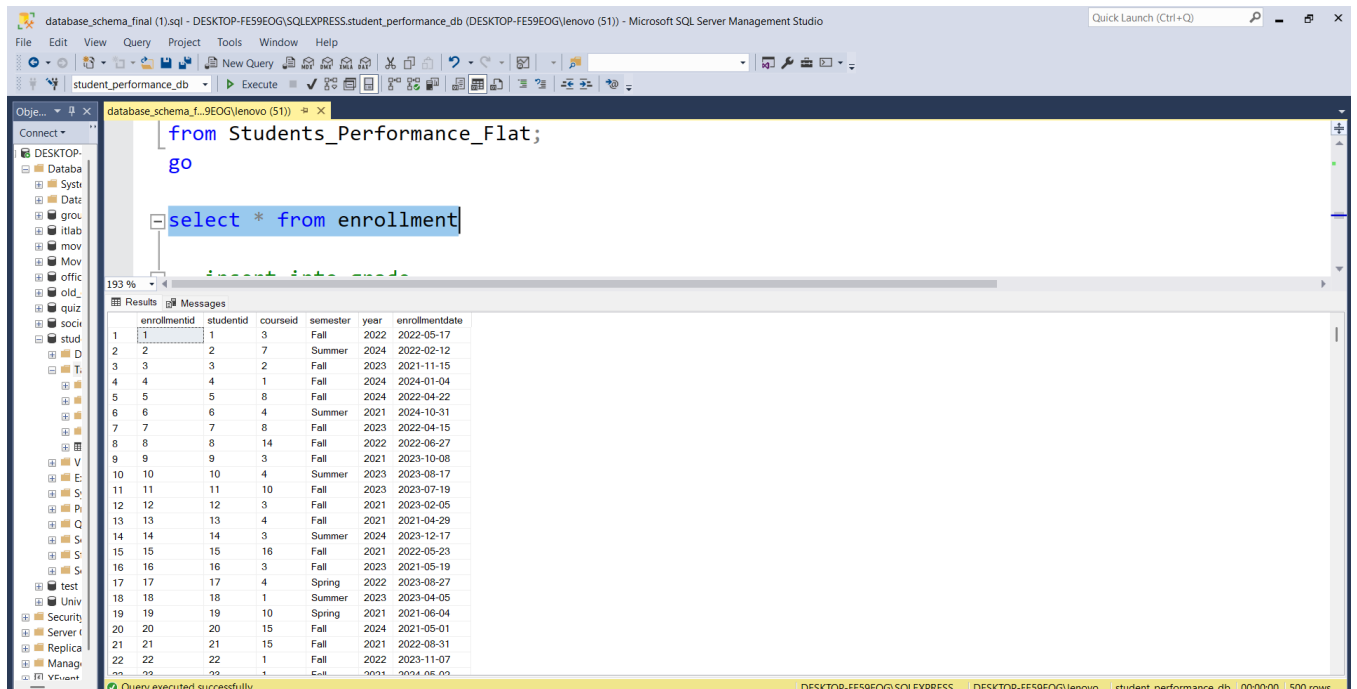


Figure 8

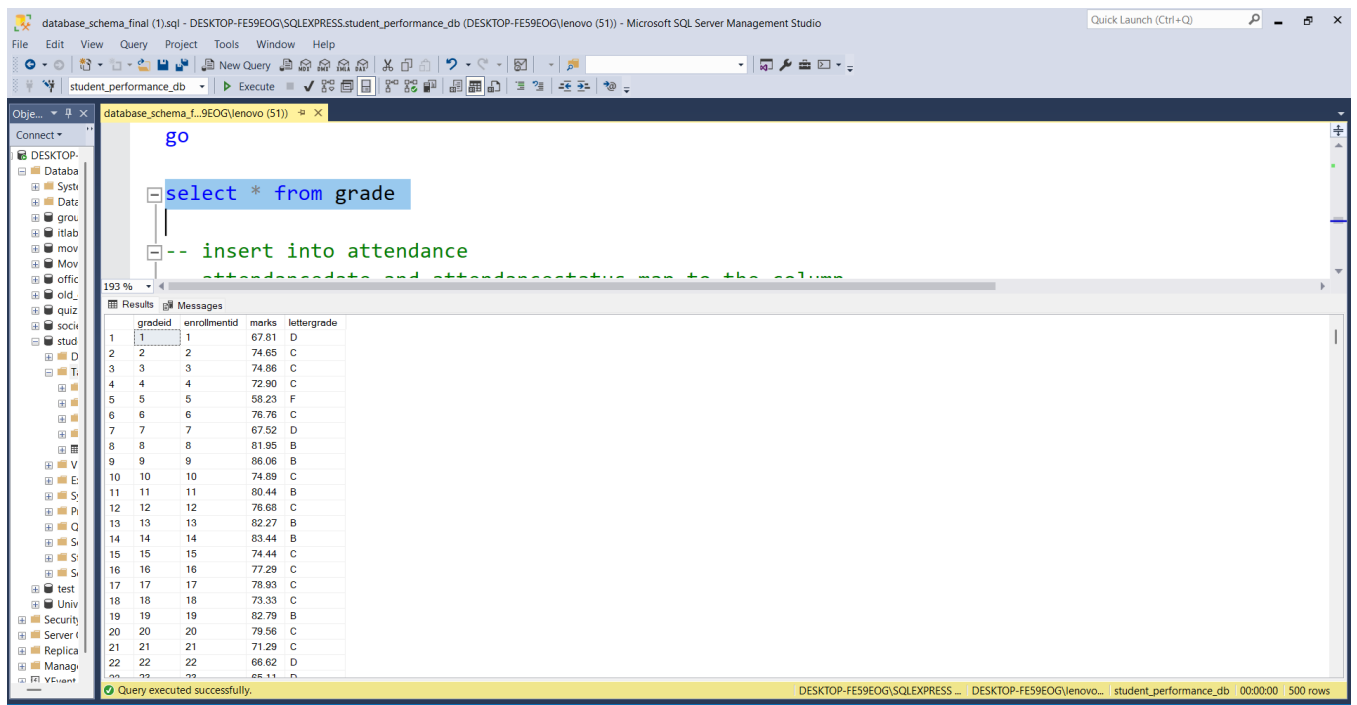


Figure 9

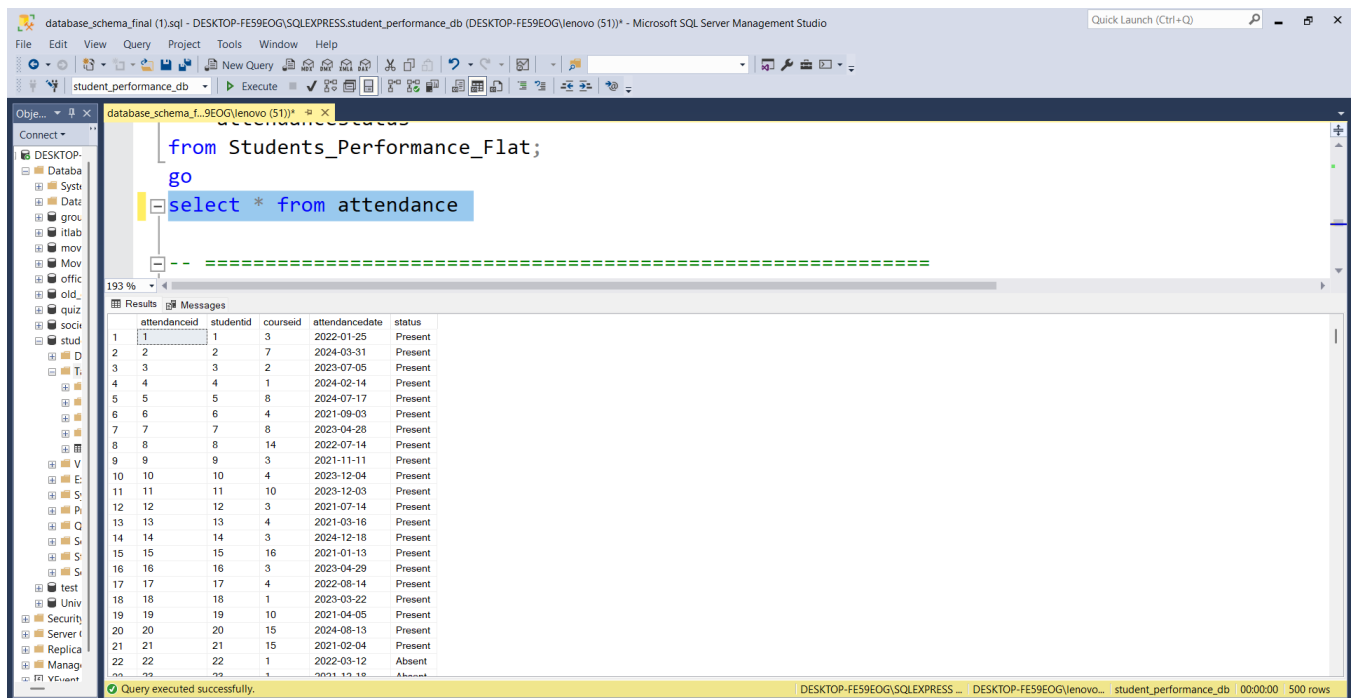


Figure 10

Screenshot 3: SELECT Query Running

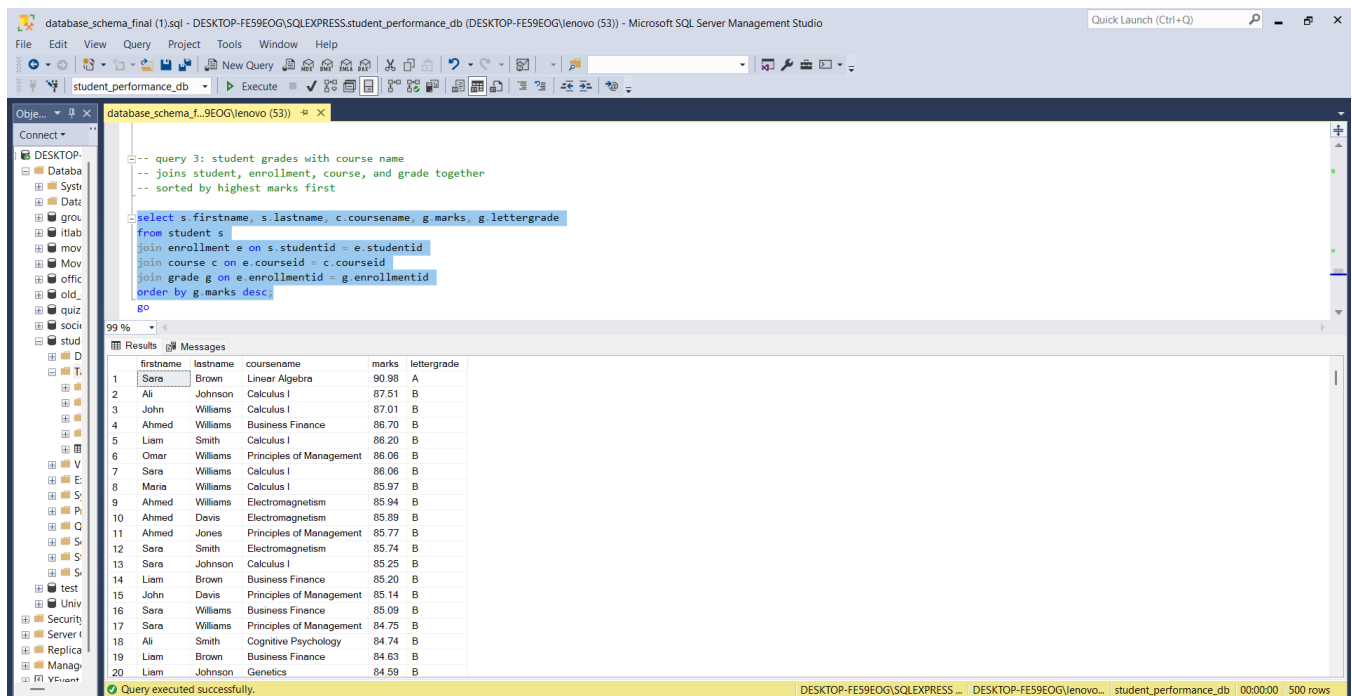


Figure 11

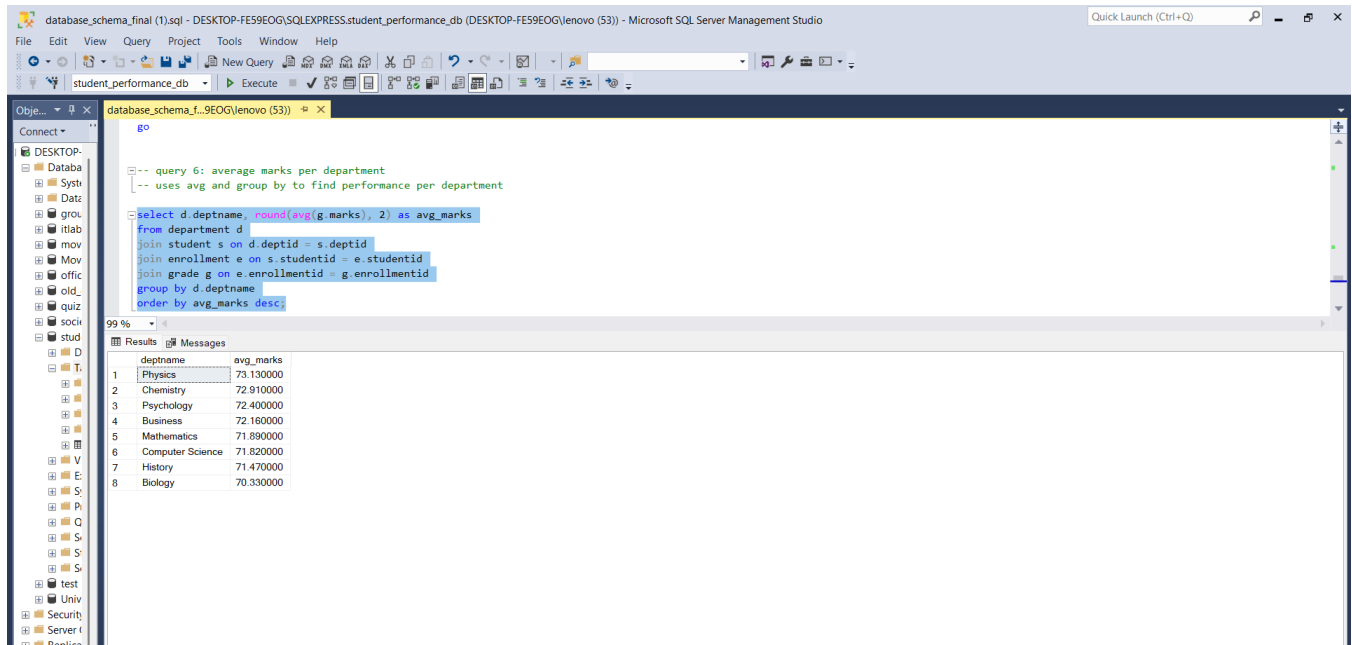


Figure 12

Deliverable 3

SQL Implementation & Testing Database Implementation Script

The database and tables were successfully created and populated.

Data from normalized tables was tested for integrity through multiple queries.

studentid	firstname	lastname	dob	email	phone	address	enrollmentdate	deptid
1	Emma	Williams	2002-10-09	student1501@university.com	+92-316-2458591	House 303, Model Town, Lahore	2022-05-17	2
2	Emma	Williams	2008-07-28	student2586@university.com	+92-324-8536477	House 302, Gulberg St, Lahore	2022-02-12	4
3	Ahmed	Smith	2004-10-11	student2653@university.com	+92-316-2556017	House 195, Garden Ave, Peshawar	2021-11-15	1
4	Ahmed	Smith	2004-04-15	student1055@university.com	+92-315-5918715	House 425, DHA Phase, Faisalabad	2024-01-04	1
5	Ahmed	Williams	2003-03-23	student705@university.com	+92-327-8606962	House 326, DHA Phase, Islamabad	2022-04-22	4
6	Emma	Jones	2002-12-20	student106@university.com	+92-348-3871230	House 274, Mall Rd, Islamabad	2024-10-31	2
7	Ahmed	Jones	2004-02-27	student589@university.com	+92-335-5491946	House 34, Mall Rd, Peshawar	2022-04-15	4
8	Ali	Brown	2001-05-27	student2468@university.com	+92-345-5408072	House 383, Model Town, Quetta	2022-06-27	7
9	Omar	Williams	2002-06-06	student2413@university.com	+92-317-3564251	House 322, Canal Rd, Quetta	2023-10-08	2
10	Liam	Williams	2007-09-03	student1600@university.com	+92-345-1192619	House 349, Garden Ave, Multan	2023-08-17	2
11	Ahmed	Williams	2001-06-11	student2464@university.com	+92-342-3997281	House 260, Garden Ave, Multan	2023-07-19	5
12	Ali	Davis	2003-06-06	student228@university.com	+92-348-6438436	House 251, Main Blvd, Karachi	2023-02-05	2
13	Emma	Smith	2001-09-03	student915@university.com	+92-315-9153566	House 418, Garden Ave, Islamabad	2021-04-29	2
14	Ali	Davis	2003-09-29	student794@university.com	+92-323-4374754	House 366, Gulberg St, Quetta	2023-12-17	2
15	Emma	Johnson	2007-05-04	student3021@university.com	+92-331-1352896	House 302, Mall Rd, Faisalabad	2022-05-23	8
16	Sara	Smith	2004-08-22	student3543@university.com	+92-314-9626108	House 122, Gulberg St, Rawalpindi	2021-05-19	2
17	John	Brown	2003-09-06	student1073@university.com	+92-316-2626229	House 338, Model Town, Peshawar	2023-08-27	2
18	Emma	Johnson	2006-03-04	student3351@university.com	+92-316-5171761	House 99, Mall Rd, Rawalpindi	2023-04-05	1
19	Emma	Williams	2004-09-12	student1744@university.com	+92-313-1247595	House 497, Garden Ave, Faisalabad	2021-06-04	5
20	Emma	Jones	2002-02-15	student1084@university.com	+92-310-7550190	House 136, Johar Town, Multan	2021-05-01	8
21	Ahmed	Williams	2001-07-14	student926@university.com	+92-347-2022698	House 383, DHA Phase, Lahore	2022-08-31	8
22	Maria	Jones	2002-02-28	student3049@university.com	+92-314-2140194	House 346, Mall Rd, Quetta	2023-11-07	1
23	Ahmed	Davis	2004-08-15	student1117@university.com	+92-336-9770838	House 162, Gulberg St, Faisalabad	2024-05-02	1
24	Liam	Davis	2004-02-10	student542@university.com	+92-339-6304573	House 476, Garden Ave, Lahore	2021-09-26	4
25	Omar	Jones	2005-07-04	student4767@university.com	+92-318-6855396	House 452, Garden Ave, Faisalabad	2022-03-13	2
26	Ahmed	Davis	2003-09-19	student501@university.com	+92-345-6022741	House 478, Garden Ave, Islamabad	2024-09-01	8
27	Ali	Smith	2004-10-05	student4066@university.com	+92-331-4415796	House 352, Gulberg St, Rawalpindi	2022-08-01	2
28	John	Smith	2005-10-11	student3333@university.com	+92-310-6596101	House 395, Canal Rd, Multan	2023-05-18	1
29	Sara	Williams	2004-04-24	student4684@university.com	+92-344-1604451	House 428, DHA Phase, Islamabad	2021-06-04	7
30	John	Brown	2006-01-09	student486@university.com	+92-325-2724590	House 182, Model Town, Islamabad	2021-03-23	2
31	Sara	Smith	2006-02-27	student1962@university.com	+92-336-5163555	House 137, Canal Rd, Karachi	2021-02-20	2
32	Liam	Williams	2006-07-24	student393@university.com	+92-324-4740036	House 13, Mall Rd, Quetta	2023-08-01	8
33	Sara	Williams	2005-10-13	student4842@university.com	+92-344-6555772	House 481, Main Blvd, Karachi	2022-12-21	1
34	Ahmed	Smith	2006-04-22	student4866@university.com	+92-330-8321474	House 311, Garden Ave, Quetta	2024-05-06	5
35	Liam	Smith	2004-01-02	student1755@university.com	+92-333-8235969	House 36, DHA Phase, Peshawar	2023-12-01	1
36	Maria	Brown	2005-11-13	student2515@university.com	+92-328-3135534	House 99, Model Town, Quetta	2023-04-17	5

SQL Queries

Categories Tested:

Basic JOIN Queries – Verified student, instructor, and department relationships.

instructorid	firstname	lastname	deptname
1	Ahmed	Khan	Mathematics
2	Sara	Ali	Business
3	Usman	Tariq	Computer Science
4	Nadia	Hussain	Physics
5	Bilal	Mehmood	Chemistry
6	Ayesha	Raza	Biology
7	Imran	Sheikh	Psychology
8	Fatima	Malik	History
9	Alyssa	Grant	Mathematics
10	Marcus	Hill	Business
11	Fatima	Khan	Computer Science
12	Jason	Nguyen	Physics
13	Lina	Torres	Business

courseid	coursename	credits	deptname
1	Calculus I	3	Mathematics
2	Linear Algebra	3	Mathematics
3	Principles of Management	3	Business
4	Business Finance	3	Business
5	Data Structures	4	Computer Science
6	Operating Systems	4	Computer Science
7	Mechanics	3	Physics
8	Electromagnetism	3	Physics
9	Organic Chemistry	3	Chemistry
10	Inorganic Chemistry	3	Chemistry
11	Genetics	3	Biology
12	Ecology	3	Biology

- JOIN & Aggregate Queries – Used AVG, SUM, MIN, MAX, and COUNT functions for performance analytics.

Deliverable ...\myous (52)

```

381  -- Average, Minimum, and Maximum marks per course
382  SELECT c.coursename,
383         AVG(g.marks) AS avgmarks,
384         MIN(g.marks) AS minmarks,
385         MAX(g.marks) AS maxmarks
386  FROM grade g
387  JOIN enrollment e ON g.enrollmentid = e.enrollmentid
388  JOIN course c ON e.courseid = c.courseid
389  GROUP BY c.coursename
390  ORDER BY avgmarks DESC;
391
392  -- Count students per department

```

100 % 1 0

Results Messages

	coursename	avgmarks	minmarks	maxmarks
1	Organic Chemistry	75.265000	60.84	84.13
2	Mechanics	73.466521	63.44	82.50
3	Business Finance	73.005230	57.19	86.70
4	Operating Systems	72.890000	60.04	84.06
5	Social Psychology	72.876470	59.89	84.04
6	Electromagnetism	72.665294	58.23	85.94
7	Calculus I	72.123466	54.58	87.51
8	Cognitive Psychology	71.851333	59.53	84.74
9	Inorganic Chemistry	71.685555	53.55	82.92
10	Modern History	71.640000	52.24	80.05
11	Linear Algebra	71.620461	59.02	90.98
12	Principles of Management	71.508690	55.33	86.06
13	World History	71.340000	61.47	82.34
14	Genetics	71.322857	62.81	84.59
15	Data Structures	70.966842	61.04	82.68
16	Ecology	69.341428	56.14	83.63

Query executed successfully. HAREEM\SOLEXPRESS (16.0 RTM) HAREEM\myous (52)

Deliverable ...\myous (52)

```

390 ORDER BY avgmarks DESC;
391
392 -- Count students per department

```

Deliverable ...\myous (52)

```

408 -- Students scoring above 85 marks in any course
409 SELECT s.firstname, s.lastname, c.coursename, g.marks
410 FROM grade g
411 JOIN enrollment e ON g.enrollmentid = e.enrollmentid
412 JOIN student s ON e.studentid = s.studentid
413 JOIN course c ON e.courseid = c.courseid
414 WHERE g.marks > 85
415 ORDER BY g.marks DESC;
416
417 -- Students enrolled in more than 3 courses
418 SELECT s.studentid, s.firstname, s.lastname, COUNT(e.courseid) AS totalcourses
419 FROM student s

```

100 %

Results Messages

	firstname	lastname	coursename	marks
1	Sara	Brown	Linear Algebra	90.98
2	Ali	Johnson	Calculus I	87.51
3	John	Williams	Calculus I	87.01
4	Ahmed	Williams	Business Finance	86.70
5	Liam	Smith	Calculus I	86.20
6	Omar	Williams	Principles of Management	86.06
7	Sara	Williams	Calculus I	86.06
8	Maria	Williams	Calculus I	85.97
9	Ahmed	Williams	Electromagnetism	85.94
10	Ahmed	Davis	Electromagnetism	85.89
11	Ahmed	Jones	Principles of Management	85.77
12	Sara	Smith	Electromagnetism	85.74
13	Sara	Johnson	Calculus I	85.25
14	Liam	Brown	Business Finance	85.20
15	John	Davis	Principles of Management	85.14
16	Sara	Williams	Business Finance	85.09

Query executed successfully. HAREEM\SQLEXPRESS (16.0 RTM) HAREEM\myous (52)

Deliverable ...\myous (52)

```

396 GROUP BY d.deptname;
397
398 -- Total marks (SUM) obtained per student across all courses
399 SELECT s.studentid, s.firstname, s.lastname, SUM(g.marks) AS totalmarks
400 FROM student s
401 JOIN enrollment e ON s.studentid = e.studentid
402 JOIN grade g ON e.enrollmentid = g.enrollmentid
403 GROUP BY s.studentid, s.firstname, s.lastname
404 ORDER BY totalmarks DESC;
405
406 -- 3. FILTERING QUERIES
407

```

100 %

Results Messages

	studentid	firstname	lastname	totalmarks
1	305	Sara	Brown	90.98
2	98	Ali	Johnson	87.51
3	38	John	Williams	87.01
4	219	Ahmed	Williams	86.70
5	183	Liam	Smith	86.20
6	9	Omar	Williams	86.06
7	33	Sara	Williams	86.06
8	53	Maria	Williams	85.97
9	345	Ahmed	Williams	85.94
10	246	Ahmed	Davis	85.89
11	325	Ahmed	Jones	85.77
12	348	Sara	Smith	85.74
13	119	Sara	Johnson	85.25
14	288	Liam	Brown	85.20
15	364	John	Davis	85.14
16	177	Sara	Williams	85.09
17	109	Sara	Williams	84.75
18	448	Ali	Smith	84.74
19	127	Liam	Brown	84.63
20	413	Liam	Johnson	84.59
21	87	Sara	Davis	84.50
22	383	Maria	Williams	84.44

Query executed successfully. HAREEM\SQLEXPRESS (16.0 RTM) HAREEM\myous (52)

Deliverable ...\myous (52)

```

423
424 -- Students with attendance rate above 90%
425 SELECT s.studentid, s.firstname, s.lastname,
426        (SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END)*100.0 / COUNT(a.attendanceid)) AS attendance_rate
427 FROM attendance a
428 JOIN student s ON a.studentid = s.studentid
429 GROUP BY s.studentid, s.firstname, s.lastname
430 HAVING (SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END)*100.0 / COUNT(a.attendanceid)) > 90
431 ORDER BY attendance_rate DESC;
432
433 -- Attendance records marked 'Absent'
434 SELECT s.firstname, s.lastname, c.coursename, a.attendanceid, a.status

```

100 % 1 0

Ln: 432, Ch: 1

Results Messages

	studentid	firstname	lastname	attendance_rate
1	1	Emma	Williams	100.00000000000000
2	2	Emma	Williams	100.00000000000000
3	3	Ahmed	Smith	100.00000000000000
4	4	Ahmed	Smith	100.00000000000000
5	5	Ahmed	Williams	100.00000000000000
6	6	Emma	Jones	100.00000000000000
7	7	Ahmed	Jones	100.00000000000000
8	8	Ali	Brown	100.00000000000000
9	9	Omar	Williams	100.00000000000000
10	10	Liam	Williams	100.00000000000000
11	11	Ahmed	Williams	100.00000000000000
12	12	Ali	Davis	100.00000000000000
13	13	Emma	Smith	100.00000000000000
14	14	Ali	Davis	100.00000000000000
15	15	Emma	Johnson	100.00000000000000
16	16	Sara	Smith	100.00000000000000
17	17	John	Brown	100.00000000000000
18	18	Emma	Johnson	100.00000000000000
19	19	Emma	Williams	100.00000000000000
20	20	Emma	Jones	100.00000000000000
21	21	Ahmed	Williams	100.00000000000000
22	24	Liam	Davis	100.00000000000000

Query executed successfully. HAREEM\SQLEXPRESS (16.0 RTM) HAREEM\myous (52) student_performance_db

Deliverable ...\myous (52)

```

429 GROUP BY s.studentid, s.firstname, s.lastname
430 HAVING (SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END)*100.0 / COUNT(a.attendanceid)) >
431 ORDER BY attendance_rate DESC;
432
433 -- Attendance records marked 'Absent'
434 SELECT s.firstname, s.lastname, c.coursename, a.attendanceid, a.status
435 FROM attendance a
436 JOIN student s ON a.studentid = s.studentid
437 JOIN course c ON a.courseid = c.courseid
438 WHERE a.status = 'Absent';
439
440 -- 4. UPDATE and DELETE OPERATIONS

```

100 % 1 0

Results Messages

	firstname	lastname	coursename	attendanceid	status
1	Maria	Jones	Calculus I	2022-03-12	Absent
2	Ahmed	Davis	Calculus I	2021-12-18	Absent
3	John	Brown	Business Finance	2022-05-11	Absent
4	Liam	Smith	Linear Algebra	2022-12-15	Absent
5	Ali	Johnson	Principles of Management	2021-08-10	Absent
6	Ali	Jones	Linear Algebra	2021-08-24	Absent
7	Maria	Jones	Linear Algebra	2023-07-25	Absent
8	Sara	Williams	Mechanics	2024-11-04	Absent
9	Maria	Williams	Principles of Management	2021-10-12	Absent
10	Ali	Johnson	Linear Algebra	2024-06-12	Absent
11	Maria	Johnson	Business Finance	2023-10-01	Absent
12	Omar	Brown	Business Finance	2021-04-30	Absent
13	Sara	Jones	Principles of Management	2021-04-12	Absent
14	John	Johnson	Modern History	2021-03-10	Absent
15	Omar	Brown	Principles of Management	2022-01-24	Absent
16	Ahmed	Johnson	Modern History	2021-02-25	Absent
17	Sara	Smith	Calculus I	2024-03-05	Absent
18	Ahmed	Johnson	Linear Algebra	2023-06-13	Absent
19	Sara	Brown	Linear Algebra	2023-04-17	Absent
20	Ahmed	Jones	Business Finance	2021-09-01	Absent
21	Liam	Williams	Organic Chemistry	2021-06-17	Absent
22	Sara	Jones	Calculus I	2021-12-06	Absent

Query executed successfully. HAREEM\SQLEXPRESS (16.0 RTM) HAREEM\myous (52)

Deliverable...\\myous (52))*

```

434 SELECT s.firstname, s.lastname, c.coursename, a.attendancedate, a.status
435 FROM attendance a
436 JOIN student s ON a.studentid = s.studentid
437 JOIN course c ON a.courseid = c.courseid
438 WHERE a.status = 'Absent';
439
440 -- 4. UPDATE and DELETE OPERATIONS
441
442 -- Add 5 new instructors
443 INSERT INTO instructor (instructorid, firstname, lastname, email, deptid)
444 VALUES
445 (501, 'Alyssa', 'Grant', 'alyssa.grant@university.com', 1),
446 (502, 'Marcus', 'Hill', 'marcus.hill@university.com', 2),
447 (503, 'Fatima', 'Khan', 'fatima.khan@university.com', 3),
448 (504, 'Jason', 'Nguyen', 'jason.nguyen@university.com', 4),
449 (505, 'Lina', 'Torres', 'lina.torres@university.com', 2);
450 select * from instructor
451
100 % 1 0

```

Results Messages

	instructorid	firstname	lastname	email	hiredate	officenum	deptid
1	1	Ahmed	Khan	ahmed.khan@uni.edu	2010-03-15	A-101	1
2	2	Sara	Ali	sara.ali@uni.edu	2012-07-01	B-201	2
3	3	Usman	Tariq	usman.tariq@uni.edu	2009-01-20	C-301	3
4	4	Nadia	Hussain	nadia.hussain@uni.edu	2015-08-10	D-401	4
5	5	Bilal	Mehmood	bilal.mehmood@uni.edu	2011-05-05	E-501	5
6	6	Ayesha	Raza	ayesha.raza@uni.edu	2013-09-25	F-601	6
7	7	Imran	Sheikh	imran.sheikh@uni.edu	2016-02-14	G-701	7
8	8	Fatima	Malik	fatima.malik@uni.edu	2014-11-03	H-801	8
9	501	Alyssa	Grant	alyssa.grant@university.com	NULL	NULL	1
10	502	Marcus	Hill	marcus.hill@university.com	NULL	NULL	2
11	503	Fatima	Khan	fatima.khan@university.com	NULL	NULL	3
12	504	Jason	Nguyen	jason.nguyen@university.com	NULL	NULL	4
13	505	Lina	Torres	lina.torres@university.com	NULL	NULL	2

Query executed successfully. HAREEM\SQLXPRESS (16.0 RTM) HAREEM\myous (52)

Deliverable...\\myous (52))*

```

453 INSERT INTO course (courseid, coursename, credits, deptid)
454 VALUES
455 (301, 'Digital Marketing', 3, 1),
456 (302, 'Financial Analytics', 4, 2),
457 (303, 'Data Warehousing', 3, 3),
458 (304, 'Organizational Behavior', 3, 4),
459 (305, 'Business Forecasting', 3, 2),
460 (306, 'International Trade', 3, 1),
461 (307, 'Network Security', 4, 3),
462 (308, 'Human Resource Strategy', 3, 4),
463 (309, 'AI for Business', 4, 1),
464 (310, 'Operations Research', 3, 2);
465 select * from course
466
100 % 1 0

```

Results Messages

	courseid	coursename	credits	deptid
1	1	Calculus I	3	1
2	2	Linear Algebra	3	1
3	3	Principles of Management	3	2
4	4	Business Finance	3	2
5	5	Data Structures	4	3
6	6	Operating Systems	4	3
7	7	Mechanics	3	4
8	8	Electromagnetism	3	4
9	9	Organic Chemistry	3	5
10	10	Inorganic Chemistry	3	5
11	11	Genetics	3	6
12	12	Ecology	3	6
13	13	Cognitive Psychology	3	7
14	14	Social Psychology	3	7
15	15	World History	3	8
16	16	Modern History	3	8
17	301	Digital Marketing	3	1
18	302	Financial Analytics	4	2
19	303	Data Warehousing	3	3
20	304	Organizational Behavior	3	4
21	305	Business Forecasting	3	2

Query executed successfully. HAREEM\SQLXPRESS (16.0 RTM) HAREEM\myous (52)

```
Deliverable...\myous (52))* X
465      || select * from course
466
467      -- Update email domain for all instructors to standardized format
468      UPDATE instructor
469      SET email = LOWER(firstname + '.' + lastname + '@university.com');
470
471      -- Delete students with attendance lower than 50%
472      ALTER TABLE enrollment
473      DROP CONSTRAINT FK__enrollmen__stude__339FAB6E;
474
475      ALTER TABLE enrollment
476      ADD CONSTRAINT FK_enrollment_student
477      FOREIGN KEY (studentid)
478      REFERENCES student(studentid)
479      ON DELETE CASCADE;
480
481      -- 5. ADVANCED FEATURE: VIEW
482
483      -- Create a view showing student performance summary
484      CREATE VIEW vwstudentperformance AS
485
100 % 1 0
Messages
(13 rows affected)
Completion time: 2026-05-07T21:30:38.9185225+05:00
100 % No issues found
Query executed successfully. HAREEM\SQLEXPRESS (16.0 RTM) HAREEM\myous (52)
```

```
Deliverable...\myous (52))* X
482
483      -- Create a view showing student performance summary
484      || CREATE VIEW vwstudentperformance AS
485      SELECT
486      s.studentid,
487      s.firstname + '.' + s.lastname AS fullname,
488      d.deptname,
489      COUNT(DISTINCT e.courseid) AS totalcourses,
490      AVG(g.marks) AS avgmarks,
491      MIN(g.marks) AS minmarks,
492      MAX(g.marks) AS maxmarks,
493      SUM(CASE WHEN a.status = 'Present' THEN 1 ELSE 0 END)*100.0 / COUNT(a.attendanceid) AS atten
494      FROM student s
495      JOIN department d ON s.deptid = d.deptid
496      JOIN enrollment e ON s.studentid = e.studentid
497      JOIN grade g ON e.enrollmentid = g.enrollmentid
498      JOIN attendance a ON s.studentid = a.studentid
499      GROUP BY s.studentid, s.firstname, s.lastname, d.deptname;
500
501      -- Test the view
502
100 % 2 0
Messages
Commands completed successfully.
Completion time: 2026-05-07T21:34:24.0868314+05:00
100 % No issues found
Query executed successfully. HAREEM\SQLEXPRESS (16.0 RTM) HAREEM\myous (52)
```

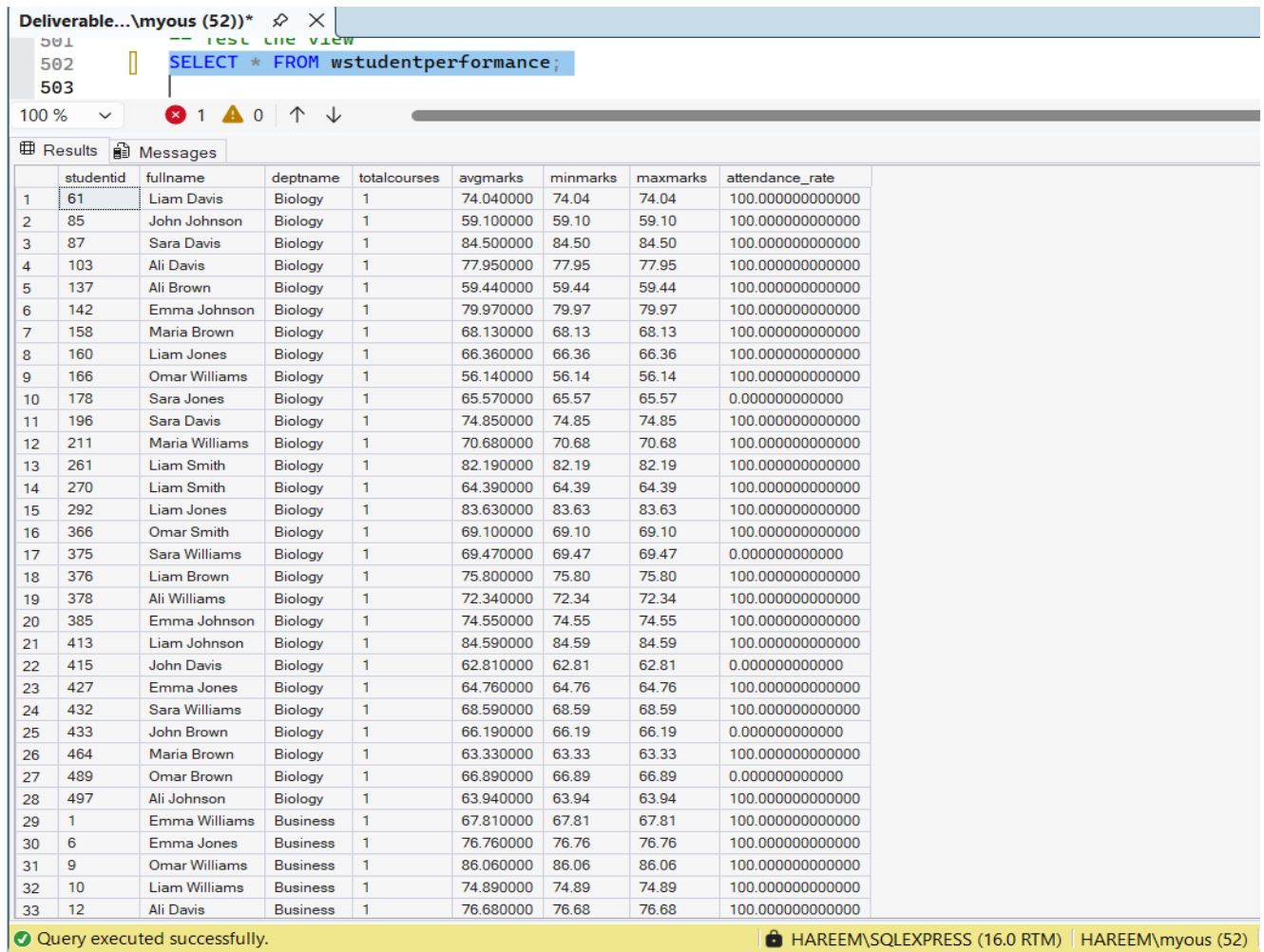
Advanced Feature

View: vwstudentperformance

A consolidated performance view linking student data, courses, grades, and attendance.

It provides:

- Average, minimum, and maximum marks
- Number of courses taken
- Department name
- Attendance rate percentage



Deliverable... \myous (52)*

501 -- test the view

502 SELECT * FROM vwstudentperformance;

503

100 % 1 0

Results Messages

	studentid	fullname	deptname	totalcourses	avgmarks	minmarks	maxmarks	attendance_rate
1	61	Liam Davis	Biology	1	74.040000	74.04	74.04	100.000000000000
2	85	John Johnson	Biology	1	59.100000	59.10	59.10	100.000000000000
3	87	Sara Davis	Biology	1	84.500000	84.50	84.50	100.000000000000
4	103	Ali Davis	Biology	1	77.950000	77.95	77.95	100.000000000000
5	137	Ali Brown	Biology	1	59.440000	59.44	59.44	100.000000000000
6	142	Emma Johnson	Biology	1	79.970000	79.97	79.97	100.000000000000
7	158	Maria Brown	Biology	1	68.130000	68.13	68.13	100.000000000000
8	160	Liam Jones	Biology	1	66.360000	66.36	66.36	100.000000000000
9	166	Omar Williams	Biology	1	56.140000	56.14	56.14	100.000000000000
10	178	Sara Jones	Biology	1	65.570000	65.57	65.57	0.000000000000
11	196	Sara Davis	Biology	1	74.850000	74.85	74.85	100.000000000000
12	211	Maria Williams	Biology	1	70.680000	70.68	70.68	100.000000000000
13	261	Liam Smith	Biology	1	82.190000	82.19	82.19	100.000000000000
14	270	Liam Smith	Biology	1	64.390000	64.39	64.39	100.000000000000
15	292	Liam Jones	Biology	1	83.630000	83.63	83.63	100.000000000000
16	366	Omar Smith	Biology	1	69.100000	69.10	69.10	100.000000000000
17	375	Sara Williams	Biology	1	69.470000	69.47	69.47	0.000000000000
18	376	Liam Brown	Biology	1	75.800000	75.80	75.80	100.000000000000
19	378	Ali Williams	Biology	1	72.340000	72.34	72.34	100.000000000000
20	385	Emma Johnson	Biology	1	74.550000	74.55	74.55	100.000000000000
21	413	Liam Johnson	Biology	1	84.590000	84.59	84.59	100.000000000000
22	415	John Davis	Biology	1	62.810000	62.81	62.81	0.000000000000
23	427	Emma Jones	Biology	1	64.760000	64.76	64.76	100.000000000000
24	432	Sara Williams	Biology	1	68.590000	68.59	68.59	100.000000000000
25	433	John Brown	Biology	1	66.190000	66.19	66.19	0.000000000000
26	464	Maria Brown	Biology	1	63.330000	63.33	63.33	100.000000000000
27	489	Omar Brown	Biology	1	66.890000	66.89	66.89	0.000000000000
28	497	Ali Johnson	Biology	1	63.940000	63.94	63.94	100.000000000000
29	1	Emma Williams	Business	1	67.810000	67.81	67.81	100.000000000000
30	6	Emma Jones	Business	1	76.760000	76.76	76.76	100.000000000000
31	9	Omar Williams	Business	1	86.060000	86.06	86.06	100.000000000000
32	10	Liam Williams	Business	1	74.890000	74.89	74.89	100.000000000000
33	12	Ali Davis	Business	1	76.680000	76.68	76.68	100.000000000000

Query executed successfully.

HAREEM\SQLXPRESS (16.0 RTM) HAREEM\myous (52)

Testing Results

All operations executed successfully:

- Aggregates handled correctly.

- Filtering produced the right student subsets.
- Update/Insert/Delete commands reflected instantly across related tables.
- View simplified overall analytics.
- Referential integrity maintained through normalized relationships.